

CS6510 · CODE WALK

Pipeline Analytics

Code Walk

How one accepted scan becomes a popular-items ranking

Self-Checkout System · Pipeline Architecture

Pipeline where the data already flows

Checkout path

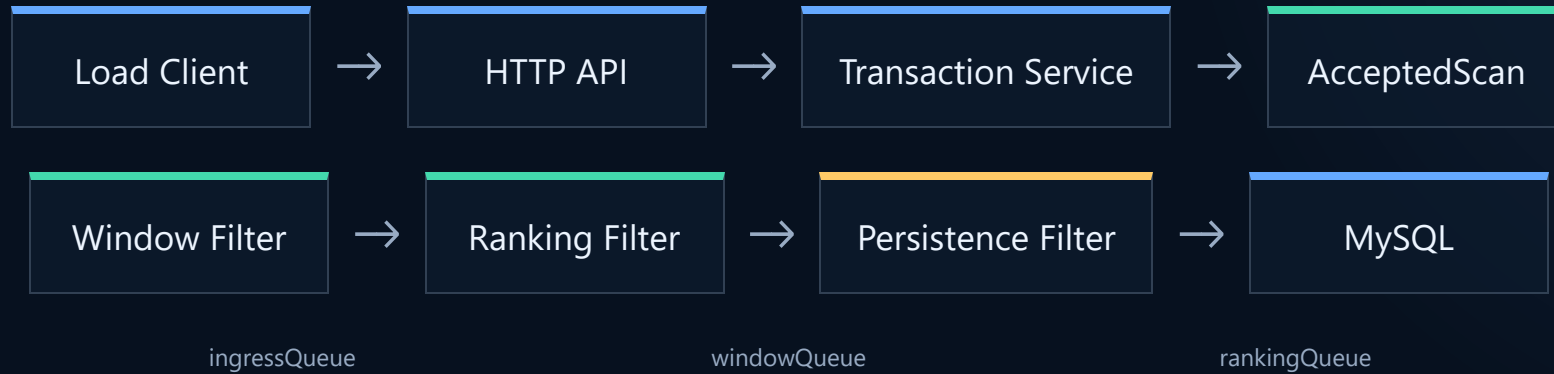
- Start a transaction
- Admit a basket scan
- Complete the purchase atomically
- Return a synchronous response

Analytics path

- Observe accepted scans
- Form fixed-size windows
- Rank the most scanned SKUs
- Persist completed rankings

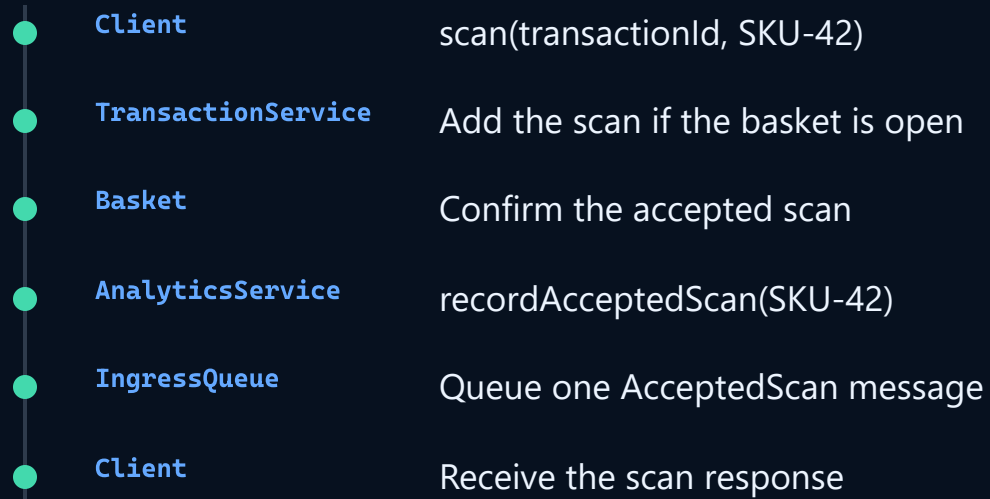
Checkout stays synchronous while analytics runs in the background.

One checkout path, three analytics stages



`Main` connects accepted scans to the analytics pipeline.

Each accepted scan creates one analytics message



The scan is accepted first. An analytics failure does not undo it.

Three stages, one task each

Window

Accepted scans
to 1,000-scan windows

queue

Ranking

Scan windows
to Top 10 items

queue

Persistence

Top 10 rankings
to the database

Independent

Each stage handles one task

In order

Queues keep windows in sequence

Clean shutdown

Every queued item finishes first

A 1,000-scan window updates every 500 scans

1-500	501-1000	1001-1500	1501-2000
Emission		Window bounds	
Scan 1000		1-1000	
Scan 1500		501-1500	
Scan 2000		1001-2000	

Windows overlap by 500 scans and use only 1,000 storage slots.

```
1 static final int WINDOW_SIZE = 1_000;
2 static final int SLIDE_INTERVAL = 500;
3
4 static int ringIndex(long scanNumber) {
5     return (int) ((scanNumber - 1) % WINDOW_SIZE);
6 }
7
8 static boolean isEmissionEligible(long accepted) {
9     return accepted ≥ WINDOW_SIZE
10         && (accepted - WINDOW_SIZE)
11             % SLIDE_INTERVAL == 0;
12 }
```

Same scans, same Top 10

Example window

```
1  A, B, A, C, A, B
```

Ranked output

```
1  1  A   3 scans
2  2  B   2 scans
3  3  C   1 scan
```

When counts tie, the SKU name decides the order.

```
1  Map<String, Long> counts = new HashMap<>();
2  for (String sku : snapshot.skus())
3      counts.merge(sku, 1L, Long::sum);
4
5  List<Map.Entry<String, Long>> ordered =
6      new ArrayList<>(counts.entrySet());
7
8  ordered.sort(
9      Map.Entry.<String, Long>
10         comparingByValue(Comparator.reverseOrder())
11         .thenComparing(Map.Entry.comparingByKey()));
```

Each save contains one complete ranking



How writes work

- Windows remain in order
- The window and its items are written together
- Retries do not create duplicates
- Readers see only completed rankings

Database error: the same ranking retries after a delay.

A long database outage can build up queued work.

The API returns the latest saved ranking

```
1 GET /analytics/popular-items?limit=10
2   |
3   ▼
4 AnalyticsHttpHandler
5   |
6   ▼
7 AnalyticsService.latestPopularItems()
```

```
1 {
2   "windowSize": 1000,
3   "slideInterval": 500,
4   "windowStart": 501,
5   "windowEnd": 1500,
6   "items": [
7     {
8       "sku": "SKU-000001",
9       "scanCount": 117,
10      "rank": 1
11    }
12  ]
13 }
```

The API returns the latest committed database snapshot.

Scan speed stays stable with 10x more stations

Workload	Transactions/s	Items/s	Scan p99	Scan errors
10 stations · 60 s	153.5	2,144.3	0.43 ms	0%
100 stations · 120 s	159.4	3,698.4	0.46 ms	0%

999

no window yet

1000

emit 1-1000

1499

no new window

1500

emit 501-1500

Popular-items counts accepted scans. Purchase completion is measured separately.

What worked and what to improve

What worked

- Stages match the analytics workflow
- In-memory queues avoid per-scan database I/O
- Immutable messages reduce concurrency risk

Next improvements

- Replay scans after a process failure
- Limit queue growth during an outage
- Monitor queue depth and processing lag

In-memory queues avoid per-scan database I/O, but a process crash loses queued scans.